KLE Society's

KLE Technological University

**An Industry Project Report**

**On**

# Object Tracking for Tablets (MARLIN)

*Submitted in partial fulfillment of the requirement for the degree of*

**Bachelor of Engineering in**

**Computer Science and Engineering**

**Submitted By**

**Namrata Nyamagoudar**
**01FE17BCS248**

**Under the guidance of**

**Industry Guide:**
**Senior Software Developer**
**Ms. Nehashree MR**

**Department Guide:**
**Prof. Umadevi F M**

SCHOOL OF COMPUTER SCIENCE &ENGINEERING,

HUBBALLI–580 031 (India).

Academic year 2020-21

**B. V. Bhoomaraddi College Campus, Vidyanagar, Hubballi - 580031.**
**Karnataka (India)**

## SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

## CERTIFICATE

This is to certify that Internship Project entitled "**Object Tracking for Tablets(MARLIN)**" is a bonafide work carried out by the student Namrata Nyamagoudar bearing USN 01FE17BCS248 in partial fulfillment of the completion of 8th semester B. E. course during the year 2020 – 2021 at **SPOOKFISH INNOVATIONS PVT LTD**. The Industry Project report has been approved as it satisfies the academic requirement with respect to the project work prescribed for the above said course.

**Name of the Guide(s)**
Industry Guide: Ms Nehashree MR
Department Guide: Prof Umadevi F M

**Head of SoCSE**

| **Name of the examiners** | **Signature with date** |
|---|---|
| 1 ---------- ---------- | 1 ---------- ---------- |
| 2 ---------- ---------- | 2 ---------- ---------- |

# CERTIFICATE

SPOOKFISH INNOVATIONS PRIVATE LIMITED
#70, 4th cross, 8th main, J C Industrial Area, Kanakapura Road, Bengaluru 560 062, INDIA
CIN: U72200KA2015PTC079666 | GSTIN: 29AAVCS6651H1ZK

14 July 2021

## CERTIFICATE OF INTERNSHIP

**To Whom It May Concern:**

This is to certify that **Namrata Namrata Nyamagoudar** (CSE, KLE, Hubballi) has successfully completed a 4 months internship at Spookfish Innovations in June 2021.

The internship involved work in close collaboration with the core development team to carry out research and development related to computer vision and image processing. The work involved understanding and implementing various methods from research papers, and finding ways to improve on existing technology to address challenging cases.

In particular, the projects that the internship spanned include:

- Real-time correspondence inference and tracking of several moving objects in video.
- Integration and experimentation with YOLO for detection, classification and tracking.
- Assistance with validation of annotated labels and training of object detection models.

Namrata 's contribution has played a vital role towards the realisation of the above technologies and their inclusion in Spookfish's core libraries.

We wish Namrata good luck in all future endeavours and look forward to working with her again in the future.

SPOOKFISH INNOVATIONS PVT LTD

Dr Sudeep Sundaram
Technical Director

# ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of a number of individuals whose professional guidance and encouragement helped me in the successful completion of this report work.

We also take this opportunity to thank Dr. Meena S M, Professor and Coordinator, School of Computer Science and Engineering for having provided us academic environment which nurtured our practical skills contributing to the success of our project.

We sincerely thank our guide Prof Umadevi FM, School of Computer Science and Engineering for his guidance, inspiration and wholehearted co-operation during the course of completion.

We sincerely thank Dr Sudeep Sundaram, Dr Anupriya B and Ms Nehashree MR, Spookfish Innovations Pvt Ltd for their support, inspiration and wholehearted co-operation during the course of completion.

Our gratitude will not be complete without thanking our beloved parents, our seniors and our friends who have been a constant source of aspirations.

**Namrata Nyamagoudar**

# ABSTRACT

In pharmaceutical industries, tablets should be tracked on the trays before packing in order to avoid defective tablets and capsules or foreign objects being packed. As such, inspection becomes critically important in manufacturing tablets/capsules. Manually tracking tablets is a cumbersome process, is prone to errors, and also expensive, hence the computer vision systems combined with artificial intelligence are becoming more and more popular. The project mainly focuses on tracking the tablet/capsule, that is, assigning a unique track id to each tablet/capsule in the frame. The frames are captured from a camera situated on top of the tablet tray, at regular intervals. The trays are of two types V-tray and U-tray, the V-tray is used for packing of capsules due to their cylindrical shape and U-tray for tablets. The trays are also color specific, white tray for dark colored tablets and grey/black tray for light-colored tablets, this makes the detection of tablets/capsules more accurate as the tablets/capsules are more prominent. The object tracking system is required to capture frames, detect the tablets/capsules accurately and assign them unique track-ids, in order to track the tablets/capsules till they exit the frame, that is reach the end of the tray. The training data is annotated according to the YOLO format and has been previously trained for detection using YOLOv4. Upon obtaining the detection weights, the YOLOv4-DeepSORT model is trained for tracking. The efficiency of tracking depends on how good the detection is and also the frames should be continuous. Several methods were experimented but DeepSORT has given by far the best results as compared to other tracking methods.



Keywords: YOLO, YOLOv4, DeepSORT, track id, detection, tracking, weights, tablets/capsules.

| Chapter No. | Table of Contents | | | Page No. |
|---|---|---|---|---|
| | **Acknowledgement** | | | **i** |
| | **Abstract** | | | **ii** |
| | **Table of contents** | | | **iii** |
| | **List of Figures** | | | **iv** |
| | **List of Tables** | | | **v** |
| **1** | **Introduction** | | | **1** |
| | 1.1 | Literature Survey | | **2** |
| | 1.2 | Motivation | | **8** |
| | 1.3 | Objectives | | **8** |
| | 1.4 | Problem Definition | | **8** |
| **2** | **Requirement Analysis** | | | 9 |
| | 2.1 | Dataset Preparation | | 9 |
| | 2.2 | System Model | | 10 |
| | 2.3 | Functional Requirements | | 10 |
| | 2.4 | Non-Functional Requirements | | 11 |
| | 2.5 | Hardware Requirements | | 11 |
| | 2.6 | Software Requirements | | 11 |
| **3** | **System Design** | | | 12 |
| | 3.1 | Architecture Design | | 12 |
| | 3.2 | Dataflow Diagram | | 13 |
| | | 3.2.1 | Level 0 DFD | 14 |
| | | 3.2.2 | Detailed DFD | 14 |
| | 3.3 | Use Case Diagram | | 15 |
| **4** | **Implementation** | | | 16 |
| | 4.1 | Data preparation – MARS dataset format | | 16 |
| | 4.2 | Cosine Metric Learning | | 16 |
| | 4.3 | DeepSORT Tracker | | 18 |
| **5** | **Testing** | | | **21** |
| | 5.1 | Integration Testing | | 21 |
| | 5.2 | Acceptance Testing | | 22 |
| **6** | **Results and Discussions** | | | **23** |
| | 6.1 | Experimental setup | | 23 |
| | | 6.1.1 | Dataset | 23 |
| | | 6.1.2 | Training Details | 23 |
| | 6.2 | Results for tablets | | 24 |
| | 6.3 | Results for capsules | | 25 |
| | 6.4 | Result Analysis | | 26 |
| **7** | **Conclusion and Future Scope** | | | **27** |
| **8** | **References** | | | **28** |

# List of Figures

2.1 System Model of the Proposed System ……. 10

3.1 Architecture Design of the proposed …….…... 13

3.2 Feature Appearance vector ……...…. 13

3.3 Level 0 DFD …….…. 14

3.4 Detailed DFD …… 14

3.6 Use Case Diagram …….……. 15

4.1 DeepSORT Tracker detailed layers …… 18

6.1 Results obtained for round tablets …….. 24

6.2 Results obtained for triangular tablets…… 25

6.3 Results obtained for oval tablets ……. 25

6.4 Results obtained for capsule ……. 25

# List of Tables

2.1 Hardware Requirements ……. 11

2.2 Software Requirements …….…. 11

4.1 Overview of cosine metric learning architecture …… 17

5.1 Integration test plan for the proposed system ……. 21

5.2 Acceptance test plan for ……. 22

# 1. Introduction

All aspects of pharmaceutical production, including packaging, are tightly controlled and have regulatory requirements. When it comes to quality control, the pharmaceutical industry has led the way in terms of product consistency, reliability and traceability. Strict requirements on quality and safety have driven the demand for complete control and transparency across the entire production process. Uniformity, cleanliness, sterility and other requirements are needed to maintain a good manufacturing practice and to ensure compliance with CGMP regulations. Hazard analysis and critical control points (HACCP) is essentially required for product safety management. As such, the Tracking systems are usually required, to keep a check on drug being packaged. With major advancements in the field of computer vision, artificial intelligence and advanced camera, the automated visual inspection systems are now capable of maintaining quality control by minimizing human error and enabling instantaneous and accurate detection and removal of defective products.

The packaging of tablet/capsule require some kind of visual inspection before they are packed, either manual or mechanized. Nevertheless, defective products such as breakage of the capsule, spilling out its contents, broken tablets, deformed tablets/capsules, tainted tablets, tablet residue and so on, or foreign objects can be found on the tray during loading of the tablets/capsules, due to which the business can be affected critically and also can be biohazardous. The solution to this can be, to build a system that can swiftly detect these abnormalities and, the defective tablets/capsules and foreign objects, can be eliminated from the tray prior to packaging.

It is also required to keep a count of the number of tablets/capsules in the production tray line. Sometimes, the products can overlap and this can lead to non-uniform tablet packaging.

The high-speed cameras and customized software are capable of accurately detecting inconsistencies in tablet color, shape and size, removing defective products using a custom-built tracking and rejection mechanism.

A system must be designed to capture frames and analyze real-time video. It should be able to detect the tablets and capsules accurately with high levels of both speed and precision. Once a foreign object or defective product is found, the supervisor should immediately stop the process and eliminate the objects. Usually, instead of stopping the process, the packages are marked and rejected, that is few packages before and after the defective packages are rejected, to ensure quality control and save time.

## 1.1 Literature Survey

There are no papers regarding tablet tracking, hence I read papers and articles regarding object tracking. Object tracking has evolved over the years with respect to efficiency, speed and accuracy.

Object detection is very different from object tracking. In object detection, when an object is detected, we put a bounding box or a mask around it and we classify the object. The objective of the object detector ends here, each of the frames in a video are processed independently and various objects in the frame are identified. In an object tracker, on the other hand, the objective is to track a particular object across the entire video. For example, if the detector detects 5 tablets in the frame, the object tracker has to identify 5 separate detections and needs to track it across the subsequent frames using a unique tracking id. For this, the tracker needs to use spatio-temporal features.

Single Object Tracking is the easiest of the tracking sub-problems. The objective here is to just lock onto a single object in the image and track it until it exits the frame. It is comparatively easier as the object of interest is only one and as such it is easier to distinguish this from other objects in the frames.

In Multiple Object Tracking, we are required to lock onto every single object in the frame that is of interest and uniquely identify each one of them and track them until every object of interest exits the frame. This becomes relatively complex as compared to single object tracking as the model has to distinguish even the identical objects from another till the end of the video or till, they exit the frame.

**Traditional Methods:**

Meanshift – Meanshift also known as, mode seeking is an algorithm used in clustering and other related unsupervised problems. This method is similar to K-means, but replaces the centroid technique of calculating the cluster centers with a weighted average that gives importance to points that are closer to the mean. The objective of the algorithm does not require an optimum "K" value like K-Means. Consider we have a detection for an object in the frame and certain features of the object such as color, texture, histogram extracted from the detection. By applying the meanshift algorithm, we have a general idea of where the mode of the distribution of features lies in the current state. In the next frame, where this distribution has changed due to the movement of the object in the frame, the meanshift algorithm looks for the new largest mode and hence tracks the object.

Optical Flow – The object is tracked using the spatio-temporal image brightness variations at a pixel level, using the optical flow method. The focus here is on obtaining a displacement vector for the object to be tracked across the frames. The object tracking with optical flow depends on the following important assumptions:

1. Brightness consistency: Brightness around a small region is assumed to remain nearly constant, although the location of the region might change.
2. Spatial coherence: Neighboring points in the scene typically belong to the same surface and hence typically have similar motions.

3. Temporal persistence: Motion of a patch has a gradual change.
4. Limited motion: Points do not move very far or in a haphazard manner.

After the above conditions are satisfied, Lucas-Kanade method is used to obtain an equation for the velocity of certain points to be tracked. The Lucas–Kanade method assumes that the displacement of the image contents between two nearby instants (frames) is small and approximately constant within a neighborhood of the point $p$ under consideration. Thus the optical flow equation can be assumed to hold for all pixels within a window centered at $p$. Namely, the local image flow (velocity) vector $(V_x, V_y)$ must satisfy

$$
\begin{aligned}
&I_x(q_1)V_x + I_y(q_1)V_y = -I_t(q_1)\\
&I_x(q_2)V_x + I_y(q_2)V_y = -I_t(q_2)\\
&\vdots\\
&I_x(q_n)V_x + I_y(q_n)V_y = -I_t(q_n)
\end{aligned}
$$

where $q_1, q_2, \dots, q_n$ are the pixels inside the window, and $I_x(q_i), I_y(q_i), I_t(q_i)$ are the partial derivatives of the image $I$ with respect to position $x$, $y$ and time $t$, evaluated at the point $q_i$ and at the current time. The above equations can be written in the matrix form as,

$$
\begin{bmatrix} V_x \\ V_y \end{bmatrix} = \begin{bmatrix} \sum_i I_x(q_i)^2 & \sum_i I_x(q_i)I_y(q_i) \\ \sum_i I_y(q_i)I_x(q_i) & \sum_i I_y(q_i)^2 \end{bmatrix}^{-1} \begin{bmatrix} -\sum_i I_x(q_i)I_t(q_i) \\ -\sum_i I_y(q_i)I_t(q_i) \end{bmatrix}
$$

Where the sums run from $i$=1 to $n$.

Using the equation and some prediction techniques, a given object can be tracked throughout the video.

The algorithm can be applied to sparse scenes because it only relies on some small windows around a certain point of interest. The local information is derived, but the disadvantage of the algorithm is that if the motion amplitude of the pixel is too large and it moves outside the local window, the algorithm will not be able to find it.

Kalman Filters – The objective of Kalman Filter is to use the available detections and previous predictions to arrive at a best guess of the present state, while keeping the possibility of errors in the process. The Kalman filter works recursively, where we take current readings, to predict the current state, then use the measurements and update our predictions. So, it essentially boils down to inferring a new distribution (the predictions) from the previous state distribution and the measurement distribution. Kalman filter works best for linear systems with Gaussian processes involved. In this case the tablets always move in the same track and follow the linear realm and also, most processes and even noise in fall into the Gaussian realm. So, the problem is suited for the use of Kalman filters.

Kalman filter algorithm consists of two stages: prediction and update. Note that the terms "prediction" and "update" are often called "propagation" and "correction," respectively, in different literature.

**Deep Learning based approaches**

Deep Regression Networks (ECCV, 2016) Paper - One of the early methods that used deep learning, for single object tracking. A model is trained on a dataset consisting of videos with labelled target frames. The objective of the model is to simply track a given object from the given image crop. To achieve this, they use a two-frame CNN architecture which uses both the current and the previous frame to accurately regress on to the object.

ROLO - Recurrent Yolo (ISCAS 2016) - The Detections from YOLO (bounding boxes) are concatenated with the feature vector from a CNN based feature extractor. Now, this concatenated feature vector, which represents most of the spatial information related to the current object, along with the information on previous state is passed onto the LSTM cell. The output of the cell, now accounts for both spatial and temporal information. This simple trick

of using CNN's for feature extraction and LSTM's for bounding box predictions gave high improvements to tracking challenges.

SORT (Simple Online Real-time Tracking: It comprises of 4 core components, detection, estimation, association and track identity creation and destruction. The detections obtained from a detector model are propagated from current frame to the next frame using a linear constant velocity model with the help of Kalman Filter. When a detection is associated to a target, the detected bounding box is used to update the target state where the velocity components are optimally solved via the Kalman filter framework. In assigning detections to existing targets, each target's bounding box geometry is estimated by predicting its new location in the latest frame. The assignment cost matrix is then computed as the intersection-over-union (IOU) distance between each detection and all predicted bounding boxes from the existing targets. The assignment is solved optimally using the Hungarian algorithm. This works particularly well when one target occludes another. When objects enter and leave the image, unique identities need to be created or destroyed accordingly. For creating trackers, we consider any detection with an overlap less than IOUmin to signify the existence of an untracked object. The tracker is initialized using the geometry of the bounding box with the velocity set to zero. Since the velocity is unobserved at this point the covariance of the velocity component is initialized with large values, reflecting this uncertainty. Additionally, the new tracker then undergoes a probationary period where the target needs to be associated with detections to accumulate enough evidence in order to prevent tracking of false positives. Tracks are terminated if they are not detected for TLost frames, you can specify what the amount of frame should be for TLost. Should an object reappear, tracking will implicitly resume under a new identity.

DeepSORT – (SIMPLE ONLINE AND REALTIME TRACKING WITH A DEEP ASSOCIATION METRIC by Nicolai Wojke†, Alex Bewley, Dietrich

Paulus) Deep SORT, an extension to SORT (Simple Online Real time Tracker), Kalman filter is a crucial component in deep SORT. Given state contains 8 variables; (u,v,a,h,u',v',a',h') where (u,v) are centres of the bounding boxes, a is the aspect ratio and h, the height of the image. The other variables are the respective velocities of the variables. The variables have only absolute position and velocity factors, since we are assuming a simple linear velocity model. The Kalman filter helps factor in the noise in detection and uses prior state in predicting a good fit for bounding boxes. For each detection, we create a "Track", that has all the necessary state information. It also has a parameter to track and delete tracks that had their last successful detection long back, as those objects would have left the scene. Also, to eliminate duplicate tracks, there is a minimum number of detections threshold for the first few frames. A distance metric is used to quantify the association and an efficient algorithm to associate the data. The authors decided to use the squared Mahalanobis distance (effective metric when dealing with distributions) to incorporate the uncertainties from the Kalman filter. Thresholding this distance can give us a very good idea on the actual associations. This metric is more accurate than euclidean distance as we are effectively measuring distance between 2 distributions. Despite the effectiveness of Kalman filter, it fails in many of the real world scenarios we mentioned above, like occlusions, different viewpoints etc. So, to improve this, the authors of Deep sort introduced another distance metric based on the "appearance" of the object. That feature vector becomes our "appearance descriptor" of the object. The "Dense 10" layer will be the appearance feature vector for the given crop. Once trained, we just need to pass all the crops of the detected bounding box from the image to this network and obtain the "128 X 1" dimensional feature vector. Now, the updated distance metric will be:

$$D = Lambda * D_k + (1 - Lambda) * D_a$$

Where Dk is the Mahalanobis distance and Da is the cosine distance between the appearance feature vectors and Lambda is the weighting factor. The importance of D_a is so high, that the authors make a claim saying, they were able to achieve state of the art even with Lambda=0, ie only using Da.

## 1.2 Motivation

Visual inspection is essential in pharmaceutical manufacturing to ensure quality control, efficiency, speed and precision. In the present scenario the supervisor has to manually inspect whether a defective product or foreign object is present and trace it till it reaches the end of the tray. Since the manual inspection is prone to human error, cumbersome, and expensive, hence, mechanized computer vision systems are brought into effect. Automating this process will save time and increase the quality and productivity of packaging. Thus, it can change the face of pharmaceutical industry by enhancing the efficiency and economy of the industry.

## 1.3 Objectives

- To build a tracking system that is compatible with YOLOv4 detection
- The model should assign unique track id to each tablet/capsule in the frame and there should be no identity switches
- To accurately detect and track all tablets/capsules in the frame
- To obtain an FPS of 20-30

## 1.4 Problem Statement

Develop a real-time tablet tracking system on a high-resolution frames captured from the system camera.

# 2. Requirement Analysis

In this chapter, we present the phases involved in requirement analysis and how the

requirements quality can be improved. We have also learned about the identified functional and non-functional requirements; and hardware and software specifications of the given project. Analysis of requirements covers activities that meet the needs and conditions for implementing a new project, considering the possibility of conflicting requirements of different stakeholders. It also helps in evaluating, recording, validating, and addressing specific needs. Requirements analysis is critical for a system or software project to succeed or fail. An introduction to the Requirement Analysis provides an overview of the entire project including purpose, scope, definitions and references of the project.

## 2.1 Dataset Preparation

Preparing dataset is the first requirement to build a model. The dataset should be fairly abundant in order to include all kinds of tablets and capsules, so that the model is able to detect all kinds of tablets/capsules. To train a YOLO model this is an important step, more variations in the data better the model. The accuracy of the final result depends on the variety of the dataset and ample number of images from each variety. The MARLIN device, which is an online tablet inspection system, retrofitted to a wide range of existing bottle filling lines with minimal alterations to the existing setup, is used to capture images of the tablets/capsules which are in motion on the tray at regular intervals of time. These images are then annotated according to the YOLO format. To train the DeepSORT model, the dataset should be present in the MARS dataset format that is, the images belonging to certain type are grouped together in a folder and the individual tablets/capsules are cropped according to the annotations, this

helps in learning the features of the object more efficiently. The training dataset consisted of 17,387 images of tablets and capsules consisting of 19 variants.

## 2.2 System Model

Figure 2.1 shows the system model of the proposed system. Input image is the image that is captured from the MARLIN device camera. The series of input images are fed to object detector model and the detected objects in the frame are assigned track ids and this track id continues till the end of the frame. In the training part the images are first trained for detection using an object detector, the obtained weights are then used in DeepSORT tracker to detect the objects in the frame.

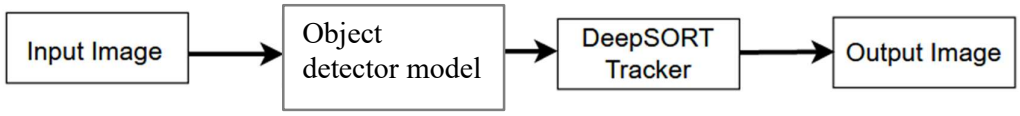


Fig 2.1: System Model of the Proposed System

## 2.3 Functional Requirements

These are the requirements demanded as basic facilities that the system should offer. All these functionalities need to be necessarily incorporated into the system as a part of the contract. These are represented or stated in the form of input to be given to the system, the operation performed and the output expected. They are basically the requirements stated which one can see directly in the final product, unlike the non-functional requirements.

The functional requirements of the project are:

1. The system shall be able to capture frames from the device.
2. The system shall be able to accurately identify the tablets/capsules in the frame.
3. The system shall be able to assign track ids to the objects in the frame.

4. The assigned track ids shall remain the same until the object exits the tray.

## 2.4 Non-Functional Requirements

These are basically the quality constraints that the system must satisfy according to the
project contract. The priority or extent to which these factors are implemented varies from one project to other. They are also called non-behavioural requirements.

The non-functional requirements of the project are:

1. The system shall be able to track objects in real-time at 20-30 frames per second.
2. The model should be easy to incorporate into the MARLIN software.

## 2.5 Hardware Requirements

| | |
|---|---|
| CPU | Intel i5/i7 (7th Gen) or equivalent |
| RAM | Minimum 8 GB |
| GPU | NVIDIA GeForce RTX 2070 SUPER or equivalent |

Table 2.1: Hardware Requirements

## 2.6 Software Requirements

| | |
|---|---|
| Operating System | Windows |
| Programming Language | Python 3 |
| Libraries | OpenCV, Numpy, Tensorflow 1.x, Scikit-Image |

Table 2.2: Software Requirements

# 3. System Design

System design is the phase that closes the gap between the problem domain and theexisting system in a practical way. This section focuses on the domain of the solution, e.g. “How is it done?” It is the stage at which the SRS document is converted into a usable format and determines how the system will work. At this stage, the complex task of program development is divided into smaller, more complex tasks, which come together to achieve the overall goal of the program development.

## 3.1 Architecture Design

Figure 3.1 shows the architectural design of the proposed system. The input video sequence or frames captured are directed to YOLO to obtain detections. Hence the object detector provides detections, the Kalman filter tracking gives the missing tracks, that is gives the direction of every individual tablet/capsule, predicts their next move in the next frame. The Hungarian algorithm associates detections to tracked objects, Hungarian algorithm achieves this by estimating the minimum distance between the object in the previous frame and its predicted locations in the next frame. As all tablets/capsules are indistinguishable in a frame, the DeepSORT model has another distance metric based on the “appearance” of the object. The network consisting of the model’s classifier with a reasonably good accuracy is taken and the final classification layer is stripped leaving behind a dense layer that produces a single feature vector. This feature vector is known as the appearance descriptor as shown in figure 3.2. After the appearance descriptor is obtained, we use the nearest neighbor queries in the visual appearance to establish the measurement to track association using the Mahalanobis distance – it is the distance between 2 points in a multivariate space. Measurement to track

association or MTA is the process of determining the relation between a measurement and an existing track.

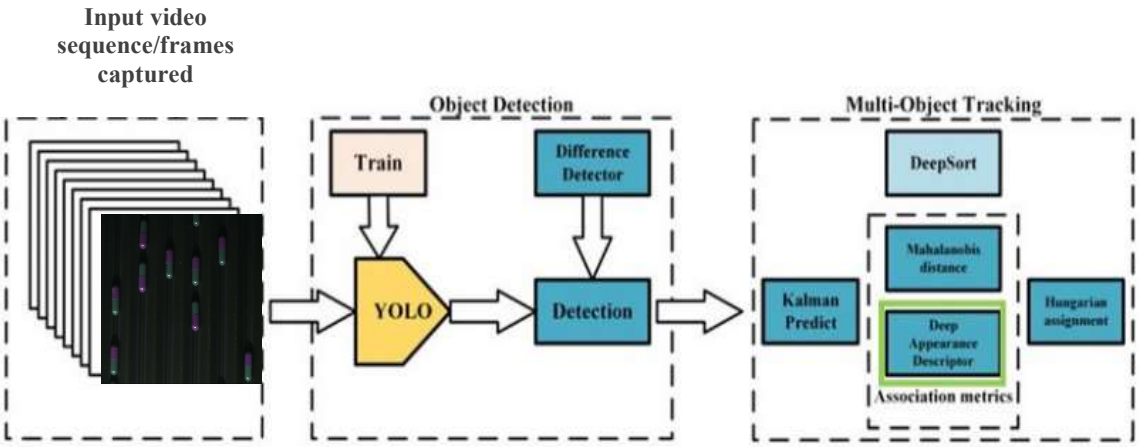


Fig 3.1: Architecture Design of the proposed

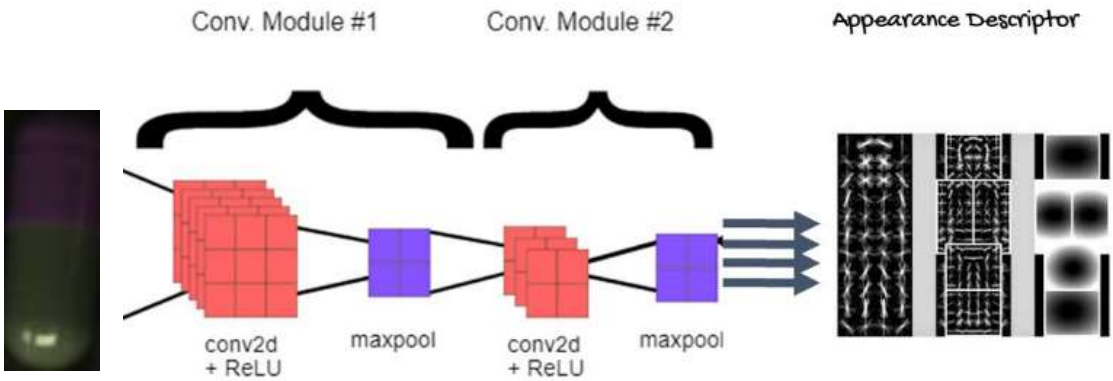


Fig 3.2: Feature Appearance vector

## 3.2 Data Design

Data design is the first design activity, which results in less complex, modular and efficient program structure. Like other software engineering activities, data design creates a model of data and/or information that is represented at a high level of abstraction (the customer or user's view of data). This data model is then refined into progressively more implementation specific representations that can be processed by the computer-based system. In many software applications, the architecture of the data will have a profound influence on the architecture of the software that must process it.

### 3.2.1 Data Flow Diagram – Level 0 DFD

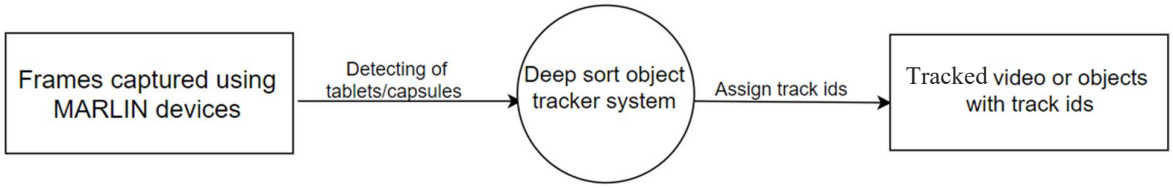


Figure 3.3: Level 0 DFD

Figure 3.3 shows the Level 0 Data Flow Diagram. It is a data flow diagram that shows entire pipeline at an abstract level. The frames captured using a marlin device are fed to the deepsort object tracker system which assigns track ids and generate the tracked video sequence.

### 3.2.2 Detailed DFD

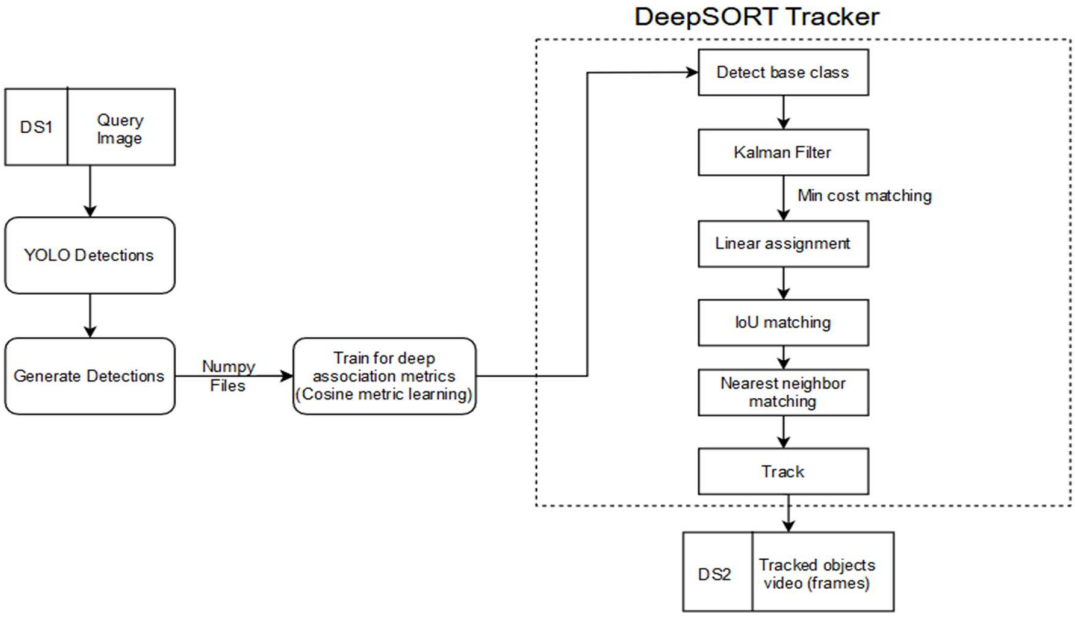


Figure 3.4: Detailed DFD

Figure 3.4 shows the detailed DFD of the proposed system. Here the images captured are first trained to generate YOLO detections, then are converted to numpy files. Then they are trained for deep association metric using cosine metric learning. Then images are fed to the DeepSORT tracker where

first the objects present in the frame are classified into their respective classes, then the next location is predicted using Kalman filter. Further, tarck id is assigned using minimum cost matching that is the linear assignment, then bounding boxes are put and IoU matching is done and the accuracy is found using the nearest neighbor matching. Finally, tracked frames are stored or a video sequence is obtained.

## 3.3 Use Case Diagram

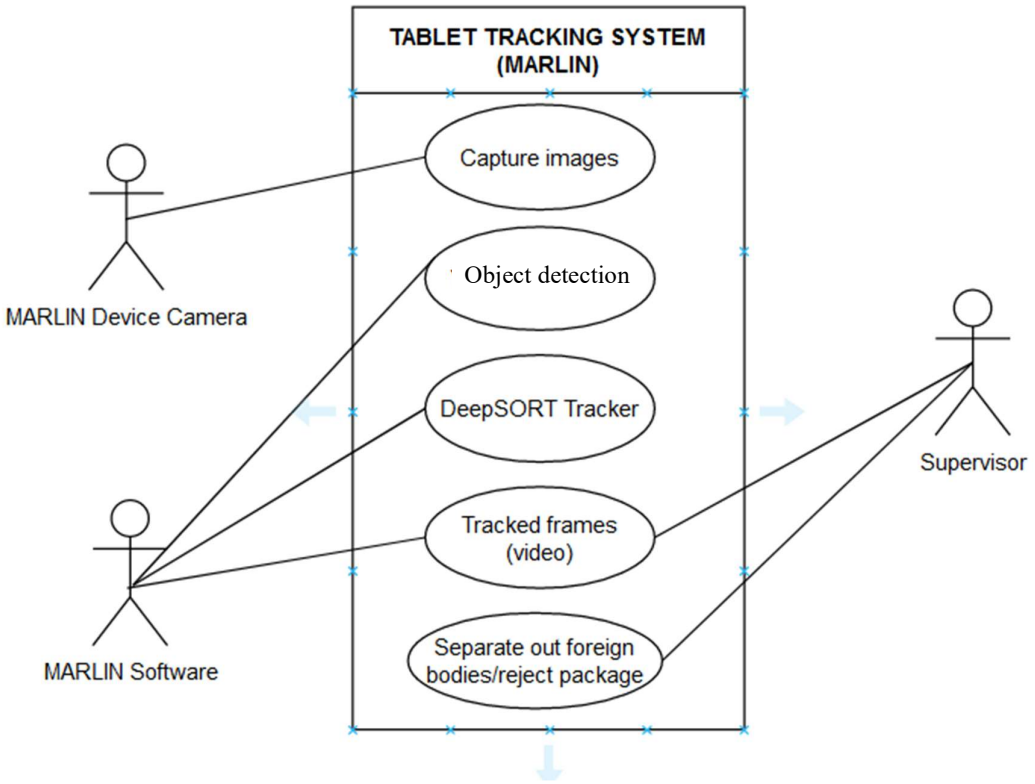


Figure 3.6: Use Case Diagram

# 4. Implementation

This chapter gives a brief description about implementation details of the system by describing each component.

## 4.1 Data preparation – MARS dataset format

As the dataset for tablets and capsules is not readily available, the first and foremost step is to prepare the available dataset of images according to the MARS dataset format. The first step is to group different variants of the images together into separate folders, then crop the individual images according to their annotations respectively. The next step is to assign these images to respective variant folders.

## 4.2 Cosine Metric Learning

Metric learning aims to construct an embedding where two extracted features corresponding to the same identity are likely to be closer than features from different identities. This is done in order to differentiate one tablet or capsule from another as they are identical to each other, hence are prone to identity switches. The cosine metric learning presents a method for learning such a feature space where the cosine similarity is effectively optimized through a simple re-parametrization of the conventional softmax classification regime. At test time, the final classification layer can be stripped from the network to facilitate nearest neighbor queries on unseen individuals using the cosine similarity metric. This approach presents a simple alternative to direct metric learning objectives such as siamese networks that have required sophisticated pair or triplet sampling strategies in the past.

The network architecture used in cosine metric learning experiments is relatively shallow to allow for fast training and inference. The architecture is summarized in Table 4.1.

| Name | Patch Size/Stride | Output Size |
|---|---|---|
| Conv 1 | $3 \times 3/1$ | $32 \times 128 \times 64$ |
| Conv 2 | $3 \times 3/1$ | $32 \times 128 \times 64$ |
| Max Pool 3 | $3 \times 3/2$ | $32 \times 64 \times 32$ |
| Residual 4 | $3 \times 3/1$ | $32 \times 64 \times 32$ |
| Residual 5 | $3 \times 3/1$ | $32 \times 64 \times 32$ |
| Residual 6 | $3 \times 3/2$ | $64 \times 32 \times 16$ |
| Residual 7 | $3 \times 3/1$ | $64 \times 32 \times 16$ |
| Residual 8 | $3 \times 3/2$ | $128 \times 16 \times 8$ |
| Residual 9 | $3 \times 3/1$ | $128 \times 16 \times 8$ |
| Dense 10 | | 128 |
| $\ell_2$ normalization | | 128 |

Table 4.1: Overview of cosine metric learning architecture

Input images are rescaled to128×64 and presented to the network in RGB color space. A series of convolutional layers reduces the size of the feature map to16×8 before a global feature vector of length 128 is extracted by layer Dense 10. The final *l2* normalization projects features onto the unit hypersphere for application of the cosine softmax classifier. The network contains several residual blocks that follow the pre-activation layout. The design follows the ideas of wide residual networks - All convolutions are of size 3×3 and max pooling is replaced by convolutions of stride 2. When the spatial resolution of the feature map is reduced, then the number of channels is increased accordingly to avoid a bottleneck. Dropout and batch normalization are used as means of regularization. Exponential linear units are used as activation function in all layers. As observed, the network architecture is shallow due to which the computation of appearance features is relatively faster.

The previously prepared dataset is presented to the above network. The computations are performed as given above in the network. The feature vectors are created in order to store the information of the training data. In the end, a protobuf file is generated which is later used in the DeepSORT tracker.

## 4.3 DeepSORT Tracker

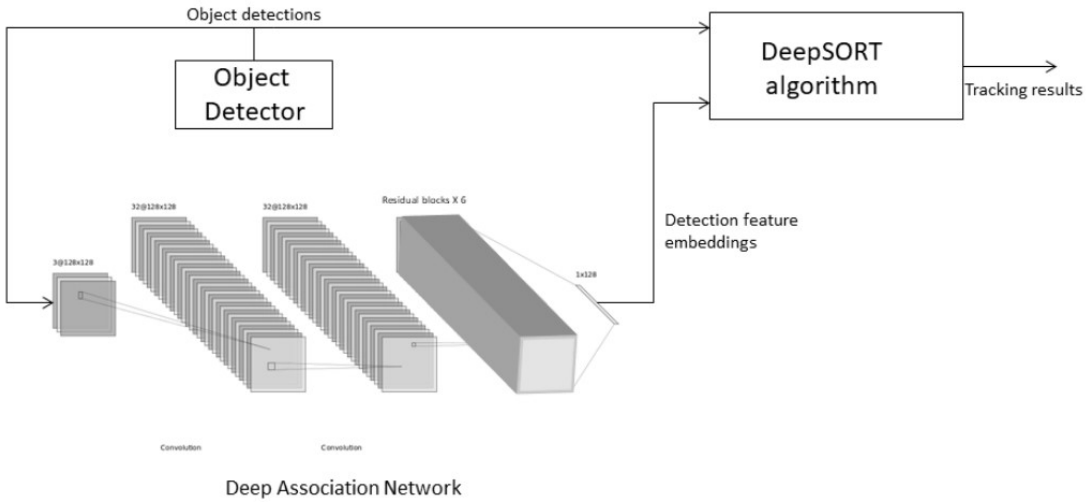


Figure 4.1: DeepSORT Tracker detailed layers

The first part of the DeepSORT algorithm is to obtain accurate detections. Accordingly, improved version of YOLO(You Only Look Once) the YOLOv4 is used. The improved features of YOLOv4 are the CSPDarknet53, which is a unique backbone that augments the learning capacity of CNN, the spatial pyramid pooling section is attached overhead CSPDarknet53 for improving the receptive field and distinguish the highly important context features, CutMix and Mosaic data augmentation, DropBlock regularization, class label smoothing, CIoU-loss, CmBN, Self-Adversarial Training, Eliminate grid sensitivity, Using multiple anchors for single ground truth, Cosine annealing scheduler, Mish activation, Cross-stage partial connections (CSP), Multi-input weighted residual connections (MiWRC). These features have made YOLOv4 more accurate and increase the frames per second by 12%. As such, the detections provided by YOLO have the accuracy of 95% in detecting tablets and capsules.

The weights obtained from YOLOv4 are converted to numpy vectors and saved as a protobuf file by saving the model. This is further used in the DeepSORT model for identifying the objects before tracking. The objects are

identified and classified into their respective classes. Next is tracking the location of the object on the tray using Kalman filter. For each track count the number of frames since the last successful measurement association. This counter is incremented during Kalman filter prediction and reset to 0 when the track has been associated with a measurement. Tracks that exceed a predefined maximum age are considered to have left the scene and are deleted from the track set. New track hypotheses are initiated for each detection that cannot be associated to an existing track. These new tracks are classified as tentative during their first three frames. During this time, we expect a successful measurement association at each time step. Tracks that are not successfully associated to a measurement within their first three frames are deleted. Next comes the Linear assignment problem. A conventional way to solve the association between the predicted Kalman states and newly arrived measurements is to build an assignment problem that can be solved using the Hungarian algorithm. Into this problem formulation it is to integrate motion and appearance information through combination of two appropriate metrics. To incorporate motion information the (squared) Mahalanobis distance between predicted Kalman states and newly arrived measurements is used. The Mahalanobis distance provides information about possible object locations based on motion that are particularly useful for short-term predictions. On the other hand, the cosine distance considers appearance information that are particularly useful to recover identities after long-term occlusions, when motion is less discriminative. To build the association problem we combine both metrics using a weighted sum. Further, matching cascade is performed for IoU association of the objects in the frame. Next using deep appearance descriptor, the cnn model whose network is obtained from cosine metric learning is used to assign track ids.

The following configurations should match with the YOLO configuration and should be tuned to obtain better results:

ANCHORS = [12,16, 19,36, 40,28, 36,75, 76,55, 72,146, 142,110, 192,243, 459,401]

STRIDES = [8, 16, 32]

XYSCALE = [1.2, 1.1, 1.05]

ANCHOR_PER_SCALE = 3

IOU_LOSS_THRESH = 0.5

For training

BATCH_SIZE = 1

INPUT_SIZE = 416

DATA_AUG = True

LR_INIT = 1e-3

LR_END = 1e-6

WARMUP_EPOCHS = 2

FISRT_STAGE_EPOCHS = 20

SECOND_STAGE_EPOCHS = 30

For testing

BATCH_SIZE = 2

INPUT_SIZE = 416

DATA_AUG = False

SCORE_THRESHOLD = 0.25

IOU_THRESHOLD = 0.5

# 5. Testing

This chapter describes the testing of proposed system. The testing is done on individual modules of the proposed system and the system as whole. Also, further the test is validated upon functional requirements specified in chapter 2.

## 5.1 Integration Testing

This section describes the testing of system as a whole. The proposed system is formed after integrating all the individual modules of the system. This test helps to validate that all the units of the system work in synchronisation. Table 5.1 shows the integration test plan for the
proposed system.

| Test ID | Input | Expected Output | Actual Output |
|---|---|---|---|
| 1 | Frame consisting of tablets | All tablets are tracked till the end of tray | All tablets are tracked till the end of tray |
| 2 | Frame consisting of capsules | All capsules are tracked till the end of tray | All capsules are tracked till the end of tray |
| 3 | Frame consisting of foreign object | Foreign object detected and tracked | Foreign object detected and tracked |
| 4 | Deformed tablet/capsule in the frame | Deformed tablet/capsule detected and tracked | Deformed tablet/capsule detected and tracked |

Table 5.1: Integration test plan for the proposed system

## 5.2 Acceptance Testing

This section concentrates on validating the results with the requirements mentioned in requirement analysis chapter. This testing is usually done to verify whether the proposed system meets customer's needs. Table 5.2 shows the acceptance test plan for the proposed system.

| Requirement ID | Test ID | Input | Expected output | Actual output |
|---|---|---|---|---|
| 2.3.2 | 1 | Image with tablets/capsules | Unique track ids for all the tablets/capsules in the frame | Unique track ids for all the tablets/capsules in the frame |
| 2.3.3 | 1 | Continuous frames containing tablets/capsules | No identity switches | No identity switches after buffer output results |

Table 5.2: Acceptance test plan for

# 6. Results and Discussion

This chapter gives a brief description about the input used and the expected output of the system. The continuous frames are given as an input to the DeepSORT model. The results using the above proposed methodology are given below.

## 6.1 Experimental setup

This section introduces the datasets, training and implementational details used within the experimentation.

### 6.1.1 Dataset

To demonstrate the proof of concept of the proposed approach, we validate the model on the dataset which is generated using the MARLIN device as no such public datasets of tablets and capsules are available. The dataset consists of 17,387 images of tablets and capsules consisting of 19 variants. Which is divided into test and train by the model. The model is tested on other various kinds of tablets and capsules.

### 6.1.2 Training Details

The training objective is optimized via Adam optimizer. The model uses Mish activation function. The training samples are resized to 416 x 416. The YOLO detection model is trained for 6000 steps which is dependent on the size of the data and cosine metric learning is trained for 10,000 steps, this can be executed until the required minimum loss is reached. The model is implemented using Tensorflow (v1.14, Python 3.7.1, CUDA 10.1 and CUDNN 7.6).

## 6.2 Results for tablets

(b) Buffer Image

(a) Buffer Image

(d) Final result

(c) Final result

Figure 6.1: Results obtained for round tablets

Figure 6.2: Results obtained for triangular tablets

Figure 6.3: Results obtained for oval tablets

## 6.3 Results for capsules

Figure 6.4: Results obtained for capsules

## 6.4 Result Analysis

The output result of DeepSORT tracker is a video sequence, in which the first two frames are called the buffer, that is no tracking appears on the objects because although the objects are rightly detected the model uses these frames to find the location of the objects and predict their progress. In my model, after the first two buffer images occur, the track id switches for upto 12 frames, the detections are not constant, as can be observed from images (a) and (b) of Figure 6.1, hence the buffer period for Tablet tracker is 12 frames, after which there is zero or very rare identity switches occurring and all the tablets are accurately detected and uniquely assigned track ids. Figures 6.1, 6.2 and 6.3 show tracking occurring for different kinds of tablets. Figure 6.3 gives the tracking for capsules, the drawback here was the frames were not continuous as such the frames showed false positives as the bounding boxes are put according to the prediction made by the model and this is done based on the previous frame, as the frames are discontinuous this leads to inaccurate detections being shown. Details of efficiency of the model are as follows:

- The model is able to achieve an inference time of 0.28 to 0.5 seconds per frame on the GPU.
- The model is able to provide 9 to 20 frames per second. The range is provided as the speed is depending upon the number of detections in the frame, more the detections, more computation time and also on the continuity of the frames, if the frames are not continuous, the model requires buffer period in order to locate the objects in the frame.
- There is very rare to no identity switches of the track ids and unique track ids are provided to the objects in the frame.

# 7. Conclusion and Future Scope

Tracking and tracing of the tablets and capsules is essential to maintain good packaging practices and to prevent defective products and foreign objects being packed, which can be hazardous. Visual inspection is essential in industrial manufacturing to ensure high production quality and high-cost efficiency. Since manual inspection by humans is slow, expensive, and error-prone, the use of automated computer vision systems is becoming increasingly popular. There are various other deep learning approaches for object tracking, but DeepSORT, the state-of-the-art method has proved efficient both in terms of speed and precision. Also, DeepSORT is compatible with the previous work done in the process that is the YOLO detection and as such can inculcated into the software. The goal is to assign track ids to every tablet/capsule in the frame and there exists no identity switching in real time scenario. As such DeepSORT has proved to be effective, as it provides the good frames per second speed, highest fps obtained being 20 and it accurately identifies every object in the frame and are assigned unique track ids and very rare identity switches. The future scope of this model is to improve the fps to at least 25-30 so as to speed-up the production unit, and also keep it in check. The buffer period of the model can be further reduced and by improving the detection, false positives can also be reduced.

# 8. REFERENCES

[1] Nicolai Wojke, Alex Bewley, Dietrich Paulus. Simple Online and Realtime Tracking with a Deep Association Metric 2017

[2] Wojke, Nicolai and Bewley, Alex (2018) *Deep Cosine Metric Learning for Person Re-Identification.* In: Proceedings - 2017 IEEE Winter Conference on Applications of Computer Vision, WACV 2017. IEEE. IEEE Winter Conference on Applications of Computer Vision (WACV), 12.-14. März 2018, Lake Tahoe, NV/CA.

[3] https://github.com/nwojke/cosine_metric_learning.git

[4] https://github.com/theAIGuysCode/yolov4-deepsort.git

[5] https://www.spookfish.vision

[6] https://augmentedstartups.medium.com/deepsort-deep-learning-applied-to-object-tracking-924f59f99104